

DAY 2 — DATA STRUCTURES & ALGORITHMS

Pattern Recognition in DSA

A Beginner-to-Interview-Level Guide

Table of Contents

1. Introduction to Pattern Recognition
 2. Why Pattern Recognition Matters
 3. The DSA Problem-Solving Pipeline
 4. Major DSA Patterns
 5. Pattern 1 — Hashing
 6. Pattern 2 — Two Sum and Hash Maps
 7. Pattern 3 — Two Pointers
 8. Pattern 4 — Sliding Window
 9. Pattern 5 — Prefix Sum
 10. Pattern 6 — Binary Search
 11. Pattern 7 — Stack
 12. Pattern 8 — Fast and Slow Pointers
 13. Pattern 9 — Tree DFS
 14. Pattern 10 — BFS
 15. Pattern 11 — Heap / Priority Queue
 16. Pattern Recognition Cheat Sheet
 17. Understanding Constraints
 18. Complexity and Pattern Selection
 19. Avoiding Premature Pattern Matching
 20. A Complete Pattern Recognition Example
 21. Pattern Recognition Framework
 22. Day 2 Practice Strategy
 23. Final Mental Model
 24. Day 2 Summary
-

1. Introduction to Pattern Recognition

One of the biggest differences between a beginner and an experienced software engineer solving DSA problems is not necessarily how much code they can write.

The difference is often their ability to recognize the **underlying structure of a problem**.

Suppose an interviewer gives you:

Given an array of integers, determine whether two numbers add up to a target value.

A beginner may think:

"I need to write some loops and figure it out."

An experienced problem solver may immediately think:

"This resembles Two Sum. I need fast lookup. A hash map can reduce the search from $O(n^2)$ to $O(n)$."

This ability is called **pattern recognition**.

Definition

Pattern recognition in DSA is the ability to identify the underlying algorithmic technique or data structure that is appropriate for solving a given problem.

The important point is that you are not memorizing individual problems.

You are learning to recognize families of problems.

For example:

```
Two Sum
3Sum
4Sum
Subarray Sum
Longest Substring
Minimum Window
Merge Intervals
Number of Islands
Climbing Stairs
Coin Change
```

These problems look different.

However, many of them can be solved using a relatively small collection of reusable patterns.

2. Why Pattern Recognition Matters

There are thousands of DSA problems.

It is impossible to memorize the solution to every possible interview question.

Instead, interview preparation should teach you to recognize common structures.

For example:

"Have I seen this value before?"

↓

Hash Set

"Find a pair that satisfies a condition"

↓

Hash Map / Two Pointers

"Contiguous substring/subarray"

↓

Sliding Window / Prefix Sum

"Sorted or monotonic search space"

↓

Binary Search

"Nested or matching elements"

↓

Stack

"Cycle in linked list"

↓

Fast + Slow Pointers

"Level-by-level traversal"

↓

BFS

"Explore deeply"



DFS

"Top K elements"



Heap

"Generate all possibilities"



Backtracking

"Optimal solution from smaller problems"

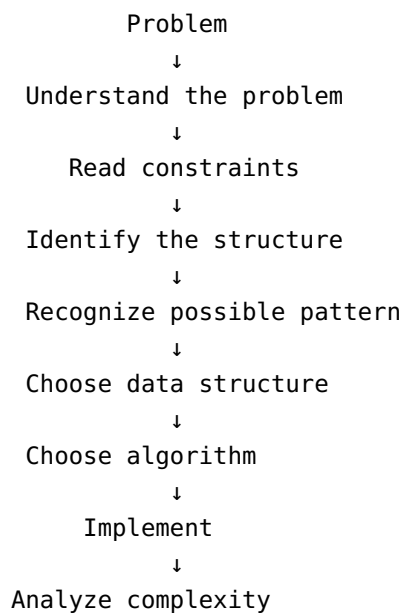


Dynamic Programming

Once you recognize these patterns, unfamiliar problems become much less intimidating.

3. The DSA Problem-Solving Pipeline

A good DSA problem-solving process should look like this:



↓
Test the solution

Each stage matters.

Do not immediately start writing code.

3.1 Understand the Problem

Before thinking about algorithms, determine:

- What is the input?
 - What is the output?
 - What exactly are we being asked to calculate?
 - Are duplicates allowed?
 - Is the input sorted?
 - Is the result required to be unique?
 - Is the order important?
 - Is the problem asking for a count, maximum, minimum, existence, or actual elements?
-

3.2 Read the Constraints

Constraints tell you how efficient your algorithm needs to be.

For example:

$n \leq 20$

may allow an exponential or backtracking solution.

But:

$n \leq 100,000$

usually means that an $O(n^2)$ algorithm is too slow.

Therefore, constraints are an important clue when selecting a pattern.

4. Major DSA Patterns

The following patterns form an important foundation for technical interviews:

Pattern	Typical Problems
Hashing	Two Sum, Contains Duplicate
Two Pointers	Valid Palindrome, 3Sum
Sliding Window	Longest Substring
Prefix Sum	Subarray Sum
Binary Search	Search in Sorted Array
Stack	Valid Parentheses
Fast & Slow Pointers	Linked List Cycle
Linked List Manipulation	Reverse Linked List
Intervals	Merge Intervals
Trees	DFS, BFS
Graphs	BFS, DFS
Heap	Top K Elements
Backtracking	Subsets, Permutations
Dynamic Programming	Climbing Stairs, Coin Change
Greedy	Jump Game

The goal of this chapter is not to master every pattern in one day.

The goal is to learn how to **recognize the patterns**.

5. Pattern 1 — Hashing

Hashing is one of the most useful techniques in DSA.

In Python, hashing is commonly represented using:

```
dict
```

and:

```
set
```

A hash-based data structure allows us to perform lookup operations efficiently.

5.1 When Should You Think About Hashing?

Ask yourself:

"Do I need to know whether I have already seen something?"

If the answer is yes, hashing is often a good candidate.

Common clues include:

```
Have I seen this before?  
Find duplicates  
Count frequencies  
Check whether an element exists  
Find a pair  
Remember previous values  
Group elements
```

6. Example — Contains Duplicate

Consider:

```
nums = [1, 2, 3, 1]
```

The question is:

Does any value appear more than once?

We need to remember the values we have already encountered.

A set is ideal for this.

6.1 Solution

```
seen = set()

for num in nums:
    if num in seen:
        return True

    seen.add(num)

return False
```

The process is:

```
1 → not seen → add 1
2 → not seen → add 2
3 → not seen → add 3
1 → already seen → duplicate
```

6.2 Complexity

```
Time Complexity:  $O(n)$ 
Space Complexity:  $O(n)$ 
```

We traverse the array once.

The set can contain up to n elements.

6.3 Recognition Rule

Remember:

"Have I seen this before?" → Think Hash Set.

7. Pattern 2 — Two Sum and Hash Maps

Consider:

```
nums = [2, 7, 11, 15]
target = 9
```

We need to find two numbers whose sum equals the target.

The naive approach is:

```
Try every pair
```

That requires:

```
 $O(n^2)$ 
```

comparisons.

But we can do better.

7.1 The Complement Idea

Suppose the current number is:

```
x
```

We need:

```
x + y = target
```

Therefore:

```
y = target - x
```

We call `y` the **complement**.

For example:

```
target = 9
current = 2

complement = 9 - 2
            = 7
```

So instead of searching through the entire array for 7, we can use a hash map.

7.2 Python Implementation

```
seen = {}

for i, num in enumerate(nums):

    complement = target - num

    if complement in seen:
        return [seen[complement], i]

    seen[num] = i
```

The dictionary stores:

```
number → index
```

7.3 Complexity

```
Time Complexity: O(n)
```

```
Space Complexity: O(n)
```

7.4 Recognition Rule

When you see:

```
pair
+
target
+
fast lookup
```

think:

Hash Map

8. Pattern 3 — Two Pointers

The Two Pointers technique uses two indices to traverse a data structure.

Typically:

```
left  →                ← right

[ 1, 2, 3, 4, 5, 6, 7 ]
```

The pointers may move toward each other, move in the same direction, or represent the boundaries of a region.

8.1 Typical Two-Pointer Situations

Look for:

```
Beginning and end
Left and right
Opposite directions
Sorted array
Pair relationships
Palindrome
```

9. Example — Valid Palindrome

Consider:

```
racecar
```

A palindrome reads the same from both directions.

Instead of reversing the entire string, compare:

```

r           r
a           a
c           c
e           e
```

This naturally suggests two pointers.

9.1 Algorithm

Initialize:

```
left = 0
right = len(s) - 1
```

Then:

```
while left < right:
    if s[left] != s[right]:
        return False

    left += 1
    right -= 1

return True
```

9.2 Complexity

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Only two pointer variables are used.

9.3 Recognition Rule

When you see:

"Compare the beginning and the end and move inward."

Think:

Two Pointers

10. Pattern 4 — Sliding Window

Sliding Window is one of the most important interview patterns.

The idea is to maintain a continuous region of an array or string.

For example:

```
[ a b c d e f g ]  
  [-----]  
  window
```

Then the window moves:

```
[ a b c d e f g ]  
    [-----]
```

Then:

```
[ a b c d e f g ]  
      [-----]
```

Instead of recalculating everything for every possible range, we maintain information about the current window.

11. Recognizing Sliding Window

Look for words such as:

```
contiguous
substring
subarray
longest
shortest
maximum
minimum
window
at most K
exactly K
```

A particularly strong signal is:

Find the longest or shortest contiguous portion satisfying a condition.

12. Example — Maximum Sum Subarray of Size K

Consider:

```
nums = [2, 1, 5, 1, 3, 2]
k = 3
```

We want the maximum sum of three consecutive elements.

First window:

```
[2, 1, 5] → 8
```

Move the window:

```
[2, 1, 5, 1, 3, 2]
  └──┬──┘
    remove 2
    add 1
```

New sum:

8 - 2 + 1 = 7

Continue.

12.1 Python Implementation

```
window_sum = sum(nums[:k])
maximum = window_sum

for right in range(k, len(nums)):

    window_sum += nums[right]
    window_sum -= nums[right - k]

    maximum = max(maximum, window_sum)
```

12.2 Complexity

Time Complexity: $O(n)$

Space Complexity: $O(1)$

The important insight is that we don't repeatedly calculate the entire window.

We update it incrementally.

13. Pattern 5 — Prefix Sum

Prefix Sum is useful when dealing with repeated sum calculations.

Suppose:

nums = [2, 4, 1, 5, 3]

We construct:

```
prefix = [0, 2, 6, 7, 12, 15]
```

The meaning is:

```
prefix[i]  
=  
sum of elements before index i
```

For example:

```
prefix[3] = 7
```

because:

```
2 + 4 + 1 = 7
```

13.1 Range Sum Formula

To calculate the sum from index `l` to index `r`:

```
sum(l, r)  
=  
prefix[r + 1] - prefix[l]
```

This allows range sums to be calculated in constant time after constructing the prefix array.

13.2 Recognition Signals

Look for:

```
range sum  
subarray sum  
sum between indices  
multiple sum queries  
cumulative sum
```

Think:

14. Pattern 6 — Binary Search

Binary Search is commonly introduced as an algorithm for searching sorted arrays.

But the deeper idea is:

Use an ordering or monotonic property to eliminate a large portion of the search space at every step.

Example:

```
[1, 3, 5, 7, 9, 11, 13]
      ↑
    middle
```

Suppose the target is:

```
11
```

Since:

```
11 > 7
```

we know everything to the left of 7 can be discarded.

We continue searching only the right half.

15. Binary Search Recognition

Look for:

```
sorted
monotonic
minimum possible
maximum possible
search for threshold
```

```
first occurrence
last occurrence
```

Binary Search is also useful when the **answer itself has an ordered search space**.

This is called:

Binary Search on the Answer

For example:

Find the minimum speed required to complete a task within K hours.

The possible speeds form an ordered range:

```
speed too slow → invalid
speed sufficient → valid
```

We can search for the boundary.

16. Pattern 7 — Stack

A stack follows:

```
LIFO
Last In
First Out
```

Imagine a stack of plates.

The last plate placed on top is the first plate removed.

Python implementation:

```
stack = []

stack.append(x)
stack.pop()
```

17. Stack Recognition

Look for:

```
matching parentheses  
nested structures  
undo  
previous element  
next greater element  
remove previous element  
monotonic
```

These are strong signals for a stack.

18. Example — Valid Parentheses

Input:

```
"({[]})"
```

We need to determine whether all brackets are correctly matched.

When we encounter:

```
(
```

we don't yet know which closing bracket will eventually match it.

We place it on the stack.

```
stack = ['(']
```

Then:

```
{
```

becomes:

```
stack = ['(', '{']
```

The most recently opened bracket must be closed first.

That is exactly LIFO behavior.

18.1 Python Implementation

```
stack = []

pairs = {
    ')': '(',
    ']': '[',
    '}': '{'
}

for char in s:
    if char in "([{":
        stack.append(char)
    else:
        if not stack or stack.pop() != pairs[char]:
            return False

return len(stack) == 0
```

18.2 Recognition Rule

Nested structures + matching + unresolved previous elements → Stack

19. Pattern 8 — Fast and Slow Pointers

Fast and Slow Pointers are especially useful for linked lists.

Use two pointers:

slow → moves one step

fast → moves two steps

For example:

```
1 → 2 → 3 → 4 → 5
    ↑      ↑
  slow   fast
```

Eventually, their relative movement can reveal useful information.

20. Common Fast/Slow Pointer Problems

Look for:

```
linked list cycle
middle of linked list
detect cycle
repeated movement
```

A classic example is detecting whether a linked list contains a cycle.

If there is a cycle:

```
1 → 2 → 3 → 4
      ↑       ↓
      ← ← ←
```

the fast pointer eventually catches the slow pointer.

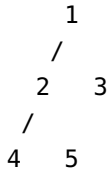
21. Pattern 9 — Tree DFS

When working with trees, ask:

"Can I solve the problem for the current node by solving the same problem for its children?"

If yes, DFS is often a natural approach.

Example:



A depth-first traversal explores one branch deeply before returning.

21.1 Recursive DFS

```
def dfs(node):  
  
    if node is None:  
        return  
  
    dfs(node.left)  
    dfs(node.right)
```

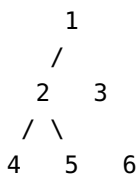
The exact placement of processing determines whether we are doing:

Preorder
Inorder
Postorder

22. Pattern 10 — BFS

Breadth-First Search explores nodes level by level.

Consider:



BFS visits:

```
1
2 3
4 5 6
```

A queue is normally used.

Python:

```
from collections import deque

queue = deque([root])

while queue:
    node = queue.popleft()

    # process node
```

22.1 BFS Recognition

Look for:

```
level order
minimum number of steps
shortest path
unweighted graph
nearest
distance
```

These often indicate BFS.

23. Pattern 11 — Heap / Priority Queue

A Heap is useful when we repeatedly need access to the smallest or largest element.

Common problem statements include:

```
top K
smallest
largest
minimum
maximum
highest priority
lowest priority
```

Python provides a heap implementation through:

```
import heapq
```

Python's `heapq` implements a **min-heap** by default.

24. Pattern Recognition Cheat Sheet

The following table should become part of your mental toolkit.

Problem Signal	Pattern
"Have I seen this before?"	Hash Set
Count frequencies	Hash Map
Pair + target	Hash Map / Two Pointers
Beginning + end	Two Pointers
Sorted array + pair	Two Pointers
Palindrome	Two Pointers
Contiguous subarray	Sliding Window / Prefix Sum
Contiguous substring	Sliding Window
Longest substring	Sliding Window
Shortest substring satisfying condition	Sliding Window
Range sum	Prefix Sum
Sorted search	Binary Search
Monotonic answer	Binary Search
Matching brackets	Stack

Problem Signal	Pattern
Next greater element	Monotonic Stack
Linked list cycle	Fast/Slow Pointers
Middle of linked list	Fast/Slow Pointers
Tree traversal	DFS / BFS
Level order	BFS
Shortest unweighted path	BFS
Top K	Heap
Repeated min/max extraction	Heap
Generate all possibilities	Backtracking
Choices + constraints	Backtracking
Overlapping subproblems	Dynamic Programming
Optimize/count ways	Dynamic Programming
Locally optimal choice	Greedy
Overlapping ranges	Intervals

25. Understanding Constraints

Constraints are one of the strongest clues in an interview problem.

Consider:

$$n \leq 20$$

An algorithm with:

$$O(2^n)$$

may still be feasible for some problems.

But consider:

$n \leq 100,000$

An:

$O(n^2)$

algorithm is generally too slow.

You should instead look for:

$O(n)$

or:

$O(n \log n)$

solutions.

26. Complexity-Based Pattern Recognition

A useful interview heuristic is:

$n \leq 20$
↓
Backtracking / Bitmask / DP

For:

$n \leq 1,000$

an:

$O(n^2)$

solution may be possible.

For:

$$n \leq 100,000$$

you generally want:

$$O(n)$$

or:

$$O(n \log n)$$

For:

$$n \geq 1,000,000$$

you should usually look for approximately:

$$O(n)$$

or another very efficient solution.

These are heuristics, not strict laws.

The actual acceptable complexity depends on:

- number of operations,
- programming language,
- time limit,
- memory limit,
- nature of the operation.

27. Do Not Pattern-Match Too Early

A major mistake beginners make is forcing a pattern simply because one keyword appears in the problem.

For example:

"The problem says subarray, therefore I should use Sliding Window."

That is not always correct.

Consider:

Find the number of subarrays whose sum equals K.

A common efficient approach is:

Prefix Sum + Hash Map

rather than an ordinary sliding window, especially when negative numbers can appear.

Therefore:

Keyword
≠
Automatic Pattern

Instead, use:

Problem
↓
Understand the exact requirement
↓
Analyze constraints
↓
Identify important properties
↓
Consider candidate patterns
↓
Select the best pattern

28. Complete Pattern Recognition Example

Consider this problem:

Given an array of integers, return the length of the longest subarray containing only unique values.

Let's solve the problem conceptually before writing code.

Step 1 — Identify the Input

The input is:

Array

Step 2 — Identify What Is Being Asked

We need:

Length

of something.

Step 3 — Identify the Important Word

The problem says:

subarray

A subarray is **contiguous**.

This is an important signal.

Step 4 — Identify the Optimization

The problem asks for:

longest

So we are looking for the largest valid contiguous region.

Step 5 — Identify the Condition

The values must be:

unique

Therefore, while maintaining our current range, we need to know which values already exist.

Step 6 — Select the Data Structure

We need fast membership checking.

Therefore:

Set / Hash Map

Step 7 — Select the Pattern

We have:

contiguous
+
longest
+
condition
+
maintain current elements

This strongly suggests:

Sliding Window + Hash Set/Hash Map

29. The Pattern Recognition Framework

For every new DSA problem, ask the following questions.

Question 1 — What is the input?

Is it:

Array?
String?
Linked List?
Tree?
Graph?
Matrix?
Number?

Question 2 — What exactly am I asked to find?

Is it:

Existence?
Count?
Maximum?
Minimum?
Length?
Actual elements?
Number of ways?
Shortest path?

Question 3 — Is the data sorted?

If yes, consider:

Binary Search
Two Pointers

Question 4 — Is the problem about a contiguous region?

If yes, consider:

Sliding Window
Prefix Sum

Question 5 — Is it asking about a pair?

Consider:

Hash Map
Two Pointers

Question 6 — Do I need fast lookup?

Consider:

Hash Set
Hash Map

Question 7 — Is there a repeated range/window?

Consider:

Sliding Window
Prefix Sum

Question 8 — Is there an ordering or monotonic property?

Consider:

Binary Search

Question 9 — Is the structure a linked list?

Consider:

Fast/Slow Pointers
Linked List Manipulation

Question 10 — Is the structure a tree or graph?

Consider:

DFS
BFS

Question 11 — Do I need all possible combinations?

Consider:

Backtracking

Question 12 — Are there overlapping subproblems?

Consider:

Dynamic Programming

Question 13 — Is there a repeated need for minimum/maximum elements?

Consider:

Heap / Priority Queue

30. The Most Important Mental Models

Pattern recognition becomes easier when you learn to associate certain descriptions with techniques.

Hashing

"Have I seen this?"

↓

Hashing

Two Pointers

"Compare both ends"

↓

Two Pointers

Sliding Window

"Contiguous + longest/shortest"

↓

Sliding Window

Prefix Sum

"Range / cumulative sum"

↓

Prefix Sum

Binary Search

"Sorted / monotonic"

↓

Binary Search

Stack

"Nested / matching"

↓

Stack

Fast and Slow Pointers

"Cycle / middle of linked list"

↓

Fast + Slow Pointers

BFS

"Level by level"

↓

BFS

DFS

"Explore deeply"

↓

DFS

Heap

"Top K / minimum / maximum"

↓

Heap

Backtracking

"Generate every possibility"

↓

Backtracking

Dynamic Programming

"Optimal solution from smaller solutions"

↓

Dynamic Programming

31. The Goal of Pattern Recognition

The objective is not:

"I know how to solve Two Sum."

The real objective is:

"When I see a new problem involving finding a pair with a target, I immediately consider hashing or two pointers, then use the constraints and structure of the problem to determine which is appropriate."

This is a much deeper skill.

You are moving from:

Memorizing solutions

to:

Recognizing problem structures

and eventually to:

32. Day 2 Practice Strategy

Do not try to solve 20 completely random problems.

Instead, solve a small number of representative problems and explicitly identify the pattern before coding.

Hashing

Practice:

1. Contains Duplicate
2. Two Sum

Before coding, write:

Pattern:
Hashing

Then explain why.

Two Pointers

Practice:

1. Valid Palindrome

Ask:

Why can I solve this from both ends?

Sliding Window

Practice:

1. Best Time to Buy and Sell Stock

2. Longest Substring Without Repeating Characters

Ask:

What is my window?
What information must I maintain?
When should the window expand?
When should it shrink?

Stack

Practice:

1. Valid Parentheses

Ask:

Why does LIFO behavior match this problem?

Binary Search

Practice:

1. Binary Search

Ask:

What property allows me to discard half of the search space?

Linked List

Practice:

1. Reverse Linked List
2. Linked List Cycle

Ask:

Which pointers do I need?
What happens to each pointer after every iteration?

33. The Pre-Coding Template

For every problem you solve during DSA preparation, write the following before writing Python code:

Problem:

Pattern:

Why this pattern?

Data structure:

Algorithm:

Time Complexity:

Space Complexity:

This forces you to think like an interviewer expects you to think.

34. Interview Thinking Process

During a real interview, your thought process should gradually become:

Read problem

↓

Clarify requirements

↓

Check constraints

↓

```
Think about brute force
↓
Identify bottleneck
↓
Recognize pattern
↓
Select data structure
↓
Develop optimized algorithm
↓
Explain it
↓
Code
↓
Test
↓
Analyze complexity
```

Notice that **coding is not the first step**.

35. Brute Force Is Still Important

Pattern recognition does not mean ignoring brute force.

In fact, brute force often helps you discover the optimized solution.

Example:

Two Sum

Brute force:

Try every pair

Complexity:

$O(n^2)$

Then ask:

"Why am I repeatedly searching for the second number?"

This leads to:

Hash Map

and:

$O(n)$

Therefore:

```
Brute Force
  ↓
Identify repeated work
  ↓
Remove repeated work
  ↓
Optimized Pattern
```

This is a powerful way to develop problem-solving ability.

36. Pattern Recognition Is Not Memorization

There is an important distinction.

Memorization

"I remember that this exact problem uses a hash map."

Pattern recognition

"The problem requires fast lookup of previously encountered values, so a hash map is appropriate."

The second skill transfers to new problems.

That is what you should aim for.

37. Day 2 Learning Objectives

By the end of Day 2, you should be able to:

- Explain what DSA pattern recognition means.
- Explain why pattern recognition is important.
- Understand the difference between memorizing solutions and recognizing patterns.
- Identify when hashing may be useful.
- Identify when to use a hash set.
- Identify when to use a hash map.
- Recognize Two Pointer problems.
- Recognize Sliding Window problems.
- Recognize Prefix Sum problems.
- Recognize Binary Search problems.
- Recognize Stack problems.
- Recognize Fast/Slow Pointer problems.
- Recognize BFS and DFS situations.
- Recognize Heap problems.
- Understand why constraints affect algorithm selection.
- Estimate whether $O(n^2)$ is acceptable.
- Avoid blindly applying patterns based on keywords.
- Explain your chosen pattern before coding.

38. Final Day 2 Summary

The most important lesson from Day 2 is:

Do not ask only "How do I solve this problem?" Ask "What kind of problem is this?"

A useful mental mapping is:

Have I seen this before?

↓

Hashing

Beginning + End

↓

Two Pointers

Contiguous + Longest/Shortest

↓

Sliding Window

Range / Cumulative Sum



Prefix Sum

Sorted / Monotonic



Binary Search

Nested / Matching



Stack

Cycle / Middle of Linked List



Fast + Slow Pointers

Level by Level



BFS

Explore Deeply



DFS

Top K / Repeated Min-Max



Heap

All Possible Choices



Backtracking

Overlapping Subproblems



Dynamic Programming

The ultimate goal is to develop this sequence of thought:

```
Problem
↓
Properties
↓
Constraints
↓
Pattern
↓
Data Structure
↓
Algorithm
↓
Complexity
↓
Implementation
```

Once this becomes natural, DSA problems stop looking like thousands of unrelated questions.

They start looking like **different variations of a relatively small number of fundamental patterns**.

That is the foundation of effective DSA interview preparation.